

STRATEX KEYHORSE TECHNICAL VALIDATION AUDIT

Evidence-Based Investor Diligence Report — stratex-habitat repository

Repository: TCCross1/stratex-habitat

Branch: copilot/stratex-tech-validation

Commit: 6eeb7605a379f5954c7343909eaede57cfc0ea8b

Generated: 2026-09-11

1. Technical Validation Report

STRATEX TECHNICAL VALIDATION REPORT

Directive: STRATEX KEYHORSE Technical Validation Directive — investor diligence audit.

Audited artifact: git repository [TCCross1/stratex-habitat](#), branch [copilot/stratex-tech-validation](#), commit [6eeb7605a379f5954c7343909eaede57cfc0ea8b](#).

Method: Direct inspection of source code, direct execution of the project's own CI-equivalent test suite against a live local MongoDB, and line-cited reading of the repository's own architecture/status documentation. No prior status report (including this repo's own [test_result.md](#)) was assumed correct; where this audit could independently re-verify a claim, it did so and reports the reproduced number. Where it could not, that is stated explicitly. See companion documents for underlying detail: [STRATEX_REPOSITORY_INVENTORY.md](#), [STRATEX_TEST_AND_BUILD_EVIDENCE.md](#), [STRATEX_FIELD_DATA_BLOCKERS.md](#).

1. Scope caveat (critical)

This audit had access to **one** repository — Habitat. ATC, Reality Foundry, Cortex, Core, Passport, and any standalone "Stratex Pro" codebase were **not present** in this environment and could not be independently built, tested, or code-reviewed. All statements below about those systems are limited to what Habitat's own integration boundary code and documentation claim about them, clearly marked as such. This is itself a material diligence finding:

a complete "Stratex platform" audit cannot be performed from this repo alone.

2. Architecture validation (item 4 of the directive)

Directive claim to validate: *Capture/DJI → ATC → Reality Foundry → reconstruction interfaces → Cortex → Core → Passport → Comprehensive Property Intelligence Report → 3D Digital Twin + Layers → standalone Stratex Pro and Stratex Habitat, with Core embedded in CENTCOM between Cortex and Passport, and Pro/Habitat standalone.*

Findings:

- Habitat is architected, in code, as a **standalone consumer application**

that never writes canonical property truth and reads it only through one versioned adapter, [backend/passport_projection.py](#) (modes: [production|development|demo|test](#); [MODE_PRODUCTION](#) uses [HttpProjectionProvider](#), which calls [HABITAT_PROJECTION_BASE_URL](#) and

fails closed with no fallback to seed/demo data if that URL is unset — [passport_projection.py:287-297](#)). This supports the directive's claim that Habitat is a standalone application relative to Passport/Core.

- **No configured, reachable Passport/Core/Cortex endpoint exists anywhere in**

this repository or its CI — [HABITAT_PROJECTION_BASE_URL](#) is not set in [.github/workflows/backend-reality.yml](#), not documented with a real value anywhere, and the production provider path was never exercised in this audit's test run. The claimed upstream chain (Capture/DJI → ATC → Reality Foundry → Cortex → Core → Passport) is not evidenced by this repository in any form beyond documentation prose

([docs/h014/H014_MASTER_ARCHITECTURE_DECISION_RECORD.md](#), ownership matrix §5) and a function in [backend/server.py:460-476](#) that is explicitly commented "Mock STRATEX Core publishing"**.

- "CENTCOM" appears in this repo exclusively as a document-header label

(# **CENTCOM DIRECTIVE H-00N: ...**) prefixing dozens of specification markdown files at repo root — it names an authoring/mission-issuing convention, not a subsystem with implementation in this codebase. **This audit cannot confirm or deny the "Core embedded inside CENTCOM between Cortex and Passport" claim** — that architecture, if real, lives outside this repository.

- "Reality Foundry" and formal reconstruction-interface terminology do not

appear in this codebase; the closest analog is Habitat's own **backend/reality/** package (scan sessions, artifacts, coordinate frames, scan guardian — 19 modules), which is a **Habitat-owned capture/ingestion boundary**, not a reconstruction engine, and produces only fixed synthetic fixture geometry today (see §6, §7).

- **Conclusion:** The directive's high-level pipeline description may be

accurate as an aspirational/enterprise architecture, but **this repository provides no first-hand evidence for the Capture→ATC→Reality Foundry→Cortex→Core→Passport chain or for CENTCOM's internal structure**. What is verified: Habitat is a standalone app; it treats Passport as an external read-only source of truth; and its own "Core" integration point is presently a mock function.

3. Subsystem classification (item 5 of the directive)

Subsystem	Classification	Basis
Home Steward (guided planning journey)	IMPLEMENTED	backend/steward.py, frontend/src/pages/HomeSteward.js ; exercised by passing integration tests (test_h013_workflow.py, test_h013_publication.py) reproduced live (§5).
Design Studio (exterior façade re-skin)	IMPLEMENTED / MOCK DATA for the AI-render path	Zones/products/recommendations are real, governed logic (backend/design.py); the Gemini "Nano Banana" re-skin render depends on the private emergentintegrations package and a live external LLM call, gated behind RUN_RENDER_TEST=1 and skipped in every observed run (§5).
Homeowner Architect projects / PIP / contractor matching	IMPLEMENTED	backend/projects.py ; full state machine (IDEA→...→SAVED_TO_PASSPORT) covered by test_projects.py , passing.
Governed price book / estimating	IMPLEMENTED / MOCK DATA (non-authoritative by design)	backend/pricebook.py is explicitly tagged authoritative:false ; it is a real, versioned, governed module but its pricing data is a seed/reference data set, not live regional market data.
Passport projection boundary	IMPLEMENTED (client adapter) / no real upstream	backend/passport_projection.py is real, tested, fails-closed code; but its production mode has never been exercised against a real Passport endpoint in this repo — the endpoint doesn't exist here.
Readiness / Build-Ready policy	IMPLEMENTED	backend/readiness_policy.py ; HARD /CONDITIONAL/WARNING/INFORMATIONAL classes with tests.

Subsystem	Classification	Basis
Redaction (contractor package)	IMPLEMENTED	backend/redaction.py , allow-list based, unit-tested.
Governed workflow + audit/passport events	IMPLEMENTED	backend/workflow.py ; immutable audit_events/passport_events collections, tested.
Reality Studio — shared spatial domain model, coordinate standard, scan sessions, artifact/object storage, Scan Quality Guardian, authz/audit	IMPLEMENTED / MOCK DATA	Real service code (backend/reality/* , 19 modules) with real governed object storage, checksums, idempotent uploads, and non-disclosure authorization — but all currently exercised only against a fixed, deterministic 12-entity/2-frame fixture or a synthetic point-cloud constant (capture_proof.py:36); never a real scan.
Room DNA / Property DNA, Interior/Exterior/Systems Studio UIs, Living Digital Home layering, Product Graph	DESIGNED / NOT IMPLEMENTED	docs/h014/README.md:1 states outright: <i>"Specification only. Not implemented."</i>
Native LiDAR capture module (Swift/RoomPlan)	IMPLEMENTED (source) / BLOCKED BY REAL FIELD DATA (execution)	ios/StratexRealityCapture compiles for the iOS Simulator SDK (per its own CI workflow's intent); its README states plainly: <i>"Build/Run status in this repository's dev/CI container: BLOCKED... can be compiled, run and used for a real device capture only on macOS + Xcode against a LiDAR device."</i> No physical capture has ever occurred per any evidence in this repo.
Physical device pilot harness (H-014B.3)	PARTIAL	Host app, protocols, redaction rules, and manual measurement sheet exist (docs/h014b3/), but they are templates/runbooks awaiting an actual pilot; test_reports/h014b3-device/ is empty/gitignored, confirming no pilot evidence has been produced yet.
Central Kentucky demo property (H-014C)	IMPLEMENTED / MOCK DATA	Deliberately and explicitly labeled: is_demo_fixture: true, dataOrigin/visualization_data_origin: demo, truthStatus: sample_only (docs/habitat/H014C_ACCEPTANCE.md). The commit message for this exact mission states production readiness is NOT READY .
Passport truth promotion (DRAFT_CANDIDATE → VERIFIED_EXISTING)	DESIGNED / NOT IMPLEMENTED (intentionally)	Governed to always reject in this repo; requires an external, unbuilt authorization step (backend/reality/audit_service.py , REALITY_TRUTH_PROMOTION_REJECTED event).
Comprehensive Property Intelligence Report	PARTIAL	The PIP/contractor-package generation pipeline is real code (projects.py , redaction.py) but has only ever been exercised on demo/fixture property data; no report has been produced from a real scan or verified for accuracy against ground truth.

Subsystem	Classification	Basis
Cortex, Core (as standalone systems), ATC, Reality Foundry proper, Stratex Pro	NOT AUDITABLE — NOT PRESENT IN THIS REPOSITORY	No code for these exists in this environment; Habitat's only "Core" touchpoint in code is an explicitly-labeled mock (server.py:460-476).

4. Mocked / simulated / stubbed / placeholder / synthetic-data inventory (item 6)

Direct grep evidence ([grep -rniE "mock|stub|simulat|synthetic|placeholder|fake_data|dummy"](#)), confirmed by reading each hit in context:

- [backend/server.py:326](#) — "Auto-route to matching contractors as leads (**mock brokerage routing**)".
- [backend/server.py:460-476](#) — function explicitly commented "**Mock STRATEX Core publishing**".
- [backend/reality/authz.py:6-9,38](#) — a dedicated "**synthetic property**" ([ref-property-h014a](#)) used as a public reference/test fixture, separate from real properties.
- [backend/reality/capture_proof.py](#) (entire file) — "Development/test ONLY... drives the REAL governed capture pipeline with clearly non-authoritative **synthetic data**"; contains a hardcoded synthetic point-cloud byte constant ([_POINTCLOUD](#), line 36) and a deterministic "PASS-grade" fake quality report ([GOOD_REPORT](#), line 44).
- [backend/seed.py:224-225](#) — "prices are **sample placeholders** (0 finding price)"; the whole file seeds demo/[sample_only](#)-tagged records.
- [backend/fixture_provider.py](#) — env-gated fixture server; unconditionally disabled when [HABITAT_ENV=production](#); every payload tagged [is_fixture=true](#), [authoritative:false](#).
- [backend/demo_property.py](#) — a single named demo property ("Central Kentucky Demonstration Home") with [truth_status="sample_only"](#) throughout.
- [frontend/src/propertyVisualization/lidarIndependentFoundations.js:2-3,46-52](#) — its own header states: "Mock fixtures for state transitions — not real capture output", and tags data with [dataOrigin: "demo"](#) / "projection_stub".
- [frontend/src/pages/HomeSteward.js](#) — a built-in "**Task 14 Failure Simulator**" UI (lines ~58-178, 1027-1031) that lets a user simulate Passport/audit/estimator outages for resiliency demos — a real feature, but explicitly simulation-only, not connected to real failure conditions.
- [backend/tests/test_projects.py:271-272](#) — test comment: "manually transition ... to **simulate** state progression."

No instance was found in this repo of synthetic/mock data being silently presented as verified truth without a label — every mock/fixture path found carries an explicit [is_fixture](#), [authoritative:false](#), [demo](#), [sample_only](#), or "non-authoritative" marker in the code or its immediate output. This is a genuine positive finding: **the truth-labeling discipline described in the architecture docs is actually implemented in code**, not just documented.

5. Test and build evidence summary (item 3; full detail in

[STRATEX_TEST_AND_BUILD_EVIDENCE.md](#))

Check	Result
Backend deps (as pinned)	FAIL — 1 private package (emergentintegrations) unavailable outside the vendor's own environment
Backend deps (CI's own exclusion method)	PASS
Backend integration test suite	PASS — 338 passed, 1 skipped, 0 failed (independently executed against local MongoDB 7.0, this session)
Backend reality coverage (process-local)	49% (informational; CI's authoritative combined-process gate of ≥83% was not reproduced)
Frontend install/build/tests	BLOCKED — no network access to required package host in this sandbox; not run
Lint/type-check	NOT CONFIGURED (tools installed as deps, never invoked)
Security/SAST scanning	NOT CONFIGURED
iOS Swift build/test workflow	Requires macOS runner, not reproducible in this sandbox; by design proves compilation only, never physical capture

6. Field-data blockers (item 7; full detail in [STRATEX_FIELD_DATA_BLOCKERS.md](#))

Everything touching real DJI/RTK/thermal capture, semantic object extraction, measurement-tolerance validation, and Passport truth-promotion is either unimplemented or backed only by a fixed synthetic fixture (12 entities, 2 frames, one demo room, one hardcoded point-cloud byte string). No M4E/M4T, RTK, point-cloud, mesh, orthomosaic, or R-JPEG thermal ingestion code exists in this repository at all.

7. Non-field-dependent remaining work (item 8; full detail in [STRATEX_FIELD_DATA_BLOCKERS.md §2](#))

Frontend build-pipeline verification, lint/type-check/security CI gates, reconciling the two duplicate state machines ([workflow.py](#) vs [projects.py](#)), building out the "specification only" H-014 Reality Studio UI layers against existing fixture data, fixing the unconditional external storage-init network call at backend startup, and rotating hardcoded demo credentials are all ordinary engineering work that does not require field data and should not be deferred until real scans arrive.

8. Audit of cross-cutting concerns (item 9)

- **Evidence lineage / audit history:** Real and reasonably strong. Immutable [audit_events](#) + [passport_events](#) collections ([backend/workflow.py](#), [backend/reality/audit_service.py](#)); the latter explicitly redacts secrets/PII before writing ([audit_service.py:12](#): forbidden-field set includes [password](#), [password_hash](#), [authorization](#), [secret](#)) and guarantees [REALITY_TRUTH_PROMOTION_REJECTED](#) is emitted even if the audit write itself fails (best-effort, per [test_result.md:510-511](#), verified against [audit_service.py](#)).
- **Property isolation:** [backend/reality/authz.py](#) implements uniform 404 non-disclosure for both nonexistent and existing-but-unauthorized properties (verified in code, not just documentation) — a genuinely good security pattern (prevents property-existence enumeration).
- **Security controls:** JWT (httpOnly cookie + bearer) with bcrypt password hashing ([server.py](#) imports, [bcrypt==4.1.3](#) pinned); storage-reference validation rejects path traversal, foreign tenant/property prefixes, URLs, and encoding tricks ([backend/reality/geometry_reference.py](#)). No SAST/dependency-vulnerability scanning is wired into CI (gap, noted above).

- **Data contracts / API boundaries:** `backend/passport_projection.py` enforces a versioned contract (`CONTRACT_VERSION`) with explicit mismatch errors (`ContractVersionError`, `TenantMismatchError`, `PropertyMismatchError`) — a real, tested contract-boundary discipline, though only ever tested against `development/demo/test` providers, never a live production counterpart.
- **Failure handling:** Circuit breaker on the HTTP projection provider (`_CB`, 3-failure threshold, 30s cooldown, `passport_projection.py:281-284`); fail-closed (never falls back to fixture data) in production mode. Good pattern, but again never exercised against a real endpoint.
- **Persistence:** MongoDB via Motor/PyMongo; the `reality` package explicitly documents "MongoDB stores metadata only — never binary chunk bytes" (`capture_upload_service.py:21`), with binary payloads routed to governed object storage — a sound separation, verified in code, including a dedicated cleanup CLI (`cleanup_binary_chunk_docs.py`) that refuses to run without an explicit `--confirm-database` match.
- **Report generation:** PIP/contractor-package generation (`projects.py`, `redaction.py`) is real and allow-list redacted, but has only ever run against demo/fixture property data — no real-report accuracy evidence exists.
- **Deployment/configuration assumptions:** Backend hard-requires `MONGO_URL`, `DB_NAME`, `JWT_SECRET` env vars at import time (`server.py:23-25, 34`) and will crash without them — reasonable fail-fast behavior, but it also unconditionally attempts to reach a specific external vendor endpoint (`integrations.emergentagent.com`) at startup regardless of configured storage provider, which this session directly observed failing (DNS resolution error) yet not crashing the app — a soft-fail that should be tightened (either honor `HABITAT_OBJECT_STORE_PROVIDER=fake` fully, or make the failure visible/alertable).
- **Third-party dependency risk:** One hard dependency (`emergentintegrations==0.2.0`) is a private, non-PyPI package not installable outside its vendor's environment — a supply-chain/portability risk flagged by the project's own CI comments. No CVE/dependency-scanning process is in place to monitor the ~30 other pinned dependencies.

9. Final investor-diligence conclusion (item 10)

A. What is demonstrably built today? A real, working FastAPI + MongoDB backend and React frontend implementing a homeowner planning/estimating/contractor-matching product ("Home Steward", "Design Studio", "Homeowner Architect"), a governed publication/audit/redaction framework, and a substantial `reality/` capture-and-storage service layer with real authorization, checksum, coordinate-frame, and idempotent-upload logic. 338 backend integration tests pass against a real MongoDB in this audit's own independent re-run. A Swift capture module and thin host app exist as reviewable, CI-compilable (on macOS) source for iPhone/iPad LiDAR capture.

B. What works using synthetic/mock data? The entire Reality Studio / spatial-geometry path (fixed 12-entity/2-frame fixture), the Passport projection boundary's non-production modes, the "Core" publish handshake (explicitly labeled mock), the governed price book (explicitly `authoritative:false`), and the one demo property/finding set used throughout QA and the H-014C mission. This is consistently and explicitly labeled in code and API responses as non-authoritative — a genuine strength.

C. What remains conventional software engineering work (no field data needed)? Fixing/verifying the frontend build pipeline in a network-enabled environment; wiring the already-installed lint/type-check tools into CI; adding security/dependency scanning; reconciling the two duplicate design-state machines; implementing the "specification only" H-014 Reality Studio UI layers (Room DNA, Living Digital Home, Product Graph) against existing fixture data; fixing the unconditional external network call at backend startup; and rotating hardcoded demo credentials.

D. What specifically cannot be responsibly completed or validated without real property scans? Any claim about measurement accuracy/tolerance, semantic object extraction, real point-cloud/mesh/orthomosaic handling, radiometric thermal data, Property Object Graph extraction from real geometry, Passport truth-promotion in practice, before/after verification workflows, and any real cross-app Pro/Habitat property synchronization (no such counterpart app exists in this

audit's scope to synchronize with).

E. Is it technically reasonable to state: "Stratex has reached the pre-field-validation boundary, where the next meaningful development stage requires real-world property capture data"? For the Habitat repository specifically: **yes, with conditions.** The governed data model, truth-labeling discipline, storage/authz/audit machinery, and native capture module source are genuinely built and tested against a strict synthetic-data boundary that the project itself refuses to promote to "verified" truth. The gating next step for the *spatial/scan* parts of this product line is indeed real captures. However, this statement **cannot be extended to the full Stratex platform** (ATC, Reality Foundry, Cortex, Core, Passport, Stratex Pro, CENTCOM) on the basis of this audit, because none of those systems' code was available to inspect. Any investor-facing claim that covers those systems must be validated against their own repositories before being asserted as fact.

Final determination — "Ready to enter funded field-validation phase"

PASS WITH CONDITIONS

Conditions:

1 Independently audit ATC/Reality Foundry/Cortex/Core/Passport/Stratex Pro

repositories with the same rigor before making any platform-wide readiness claim — this report only covers Habitat.

1 Resolve the frontend build/dependency install failure in a network-enabled

environment and obtain a real green build + test run before relying on frontend functionality claims.

1 Wire lint/type-check/security scanning into CI (tools are already pinned

as dependencies but unused).

1 Fix the unconditional external network call at backend startup and replace

the private, non-PyPI [emergentintegrations](#) dependency with a supply-chain

-safe equivalent or documented vendoring strategy.

1 Complete the H-014 "specification only" Reality Studio UI/data layers so

that, the moment real captures are available, there is a working ingestion path to receive them — do not wait for field data to start this work.

1 Execute the H-014B.3 physical device pilot (protocol already authored,

[docs/h014b3/](#)) to produce the first real measurement-tolerance evidence before any accuracy claim is made publicly.

2. Repository Inventory

STRATEX REPOSITORY INVENTORY

Prepared as part of the STRATEX KEYHORSE Technical Validation Directive. Evidence-based. No prior status report was assumed correct; every claim below was checked directly against this repository's working tree, git history, CI configuration, and a live local test execution.

1. Scope limitation (read first)

This sandboxed audit environment provides **exactly one** git repository:

Field	Value
Local path	<code>/home/runner/work/stratex-habitat/stratex-habitat</code>
Remote (origin)	<code>TCCross1/stratex-habitat</code> (formerly <code>TCCross1/Habitat</code> — see <code>docs/h013/H013_REPOSITORY_VERIFICATION.md:14</code> , confirming a prior rename)
Active branch	<code>copilot/stratex-tech-validation</code>
HEAD commit	<code>6eeb7605a379f5954c7343909eaede57cfc0ea8b</code>
HEAD subject	<code>H-014C.0 reconcile Central Kentucky demo truth and visualization</code>
HEAD author date	<code>2026-07-23</code>
Parent commit	<code>28e2119 — H-014B.3 prepare physical LiDAR device validation harness</code>

No other Stratex repository (ATC, Reality Foundry, Cortex, Core, Passport, CENTCOM, Stratex Pro) is present, cloned, or reachable from this environment. This inventory therefore cannot independently validate those systems' code, tests, or build health — only what this repository claims/implements about them. Any statement about ATC/Cortex/Core/Passport/Pro below is an inference from this repo's documentation and integration boundary code (`backend/passport_projection.py`), not a first-hand audit of those systems. This is itself a diligence finding: **the "platform" as described in the directive is not fully co-located in one auditable codebase from this vantage point.**

2. Repository identity and purpose

- **Name:** `stratex-habitat` (application title inside the code: `STRATEX HABITAT API`, `backend/server.py:182`).
- **Purpose:** A homeowner-facing web application ("Habitat") for property stewardship — home system tracking, project/estimate planning ("Home Steward", "Design Studio", "Homeowner Architect"), contractor matching/quoting, and (in progress) a scan-based "Reality Studio" for interior/exterior digital-twin capture. It positions itself as consuming — never owning — canonical property truth from an external "Passport"/"Core" system via a read-only projection boundary (`backend/passport_projection.py`).
- **Stack:** FastAPI + MongoDB (Motor/PyMongo) backend (`backend/`), Create-React-App/CRACO + Radix UI frontend (`frontend/`), a Swift Package + thin SwiftUI host app for iPhone/iPad LiDAR capture (`ios/`), and two GitHub Actions workflows (`.github/workflows/backend-reality.yml`, `.github/workflows/ios-capture.yml`).

- **Not present in this repo:** any ATC ingestion service, DJI/M4E/M4T ingestion pipeline, Reality Foundry reconstruction engine, Cortex analysis engine, Core, Passport, CENTCOM orchestration service, or a standalone "Stratex Pro" application. The repo only contains a **client-side integration boundary** (`passport_projection.py`) that expects such systems to exist elsewhere and fails closed if they don't.

3. Last meaningful development status (self-reported, verified against code)

The repository carries an internal "mission" log naming convention (`H-013`, `H-014`, `H-014A/B/B.3/C`). The most recent completed mission, per the HEAD commit message and `docs/habitat/H014C_ACCEPTANCE.md`, is:

H-014C.0 — reconciliation of a single fixture-marked "Central Kentucky Demonstration Home" so its visualization and finding copy are clearly labeled `demo / sample_only`, plus a guarded dry-run reconciliation CLI (`backend/scripts/reconcile_ky_demo.py`). The commit message itself states: *"Does not complete H-014C real existing-room twin. Physical LiDAR, RoomPlan, real-device upload, and approved Passport geometry projection remain unexecuted. Production readiness: NOT READY."*

This is a **first-party admission that the platform is pre-field-validation**, consistent with the rest of this audit's findings (Section 6 of the technical report).

4. Directory map (what actually exists)

Path	Contents
<code>backend/</code>	FastAPI app (<code>server.py</code>), Home Steward (<code>steward.py</code>), Design Studio (<code>design.py</code>), Homeowner Architect projects/PIP (<code>projects.py</code>), governed price book (<code>pricebook.py</code>), redaction (<code>redaction.py</code>), readiness/Build-Ready policy (<code>readiness_policy.py</code>), Passport projection adapter (<code>passport_projection.py</code>), demo/fixture governance (<code>fixture_provider.py</code> , <code>demo_property.py</code> , <code>seed.py</code>), governed workflow/publication (<code>workflow.py</code>), and the <code>reality/</code> package (19 modules: capture, scan sessions, artifacts, object storage, coordinate frames, scan guardian, authz, audit). 16 pytest files under <code>backend/tests/</code> .
<code>frontend/</code>	React 19 SPA; routes for <code>/twin</code> , <code>/steward</code> , <code>/design-studio</code> , <code>/quotes</code> , <code>/marketplace</code> , <code>/contractors</code> , <code>/reviews</code> , <code>/documents</code> , <code>/reports</code> , plus a dev-only <code>/reality-foundation</code> screen. 2 Jest test files.
<code>ios/</code>	<code>StratexRealityCapture</code> (Swift Package: LiDAR/RoomPlan capture module, reviewable source, not built/run in this environment — no macOS/Xcode) and <code>StratexRealityCaptureApp</code> (thin SwiftUI host app for the H-014B.3 physical device pilot).
<code>docs/h013/ ... docs/habitat/</code>	~70 markdown specification/implementation/evidence documents describing each mission (H-013 hardening, H-014 Reality Studio architecture spec, H-014A backend foundation, H-014B LiDAR capture proof, H-014B.3 physical device protocol, H-014C Kentucky demo reconciliation).
Repository root	~90 additional product/design specification markdown files (<code>HOME_STEWARD_*</code> , <code>PROJECT_*</code> , <code>DESIGN_STUDIO_*</code> , <code>ESTIMATE_*</code> , etc.) — these are product/spec documents, not verified implementation status.

Path	Contents
test_result.md	A 559-line running YAML-in-markdown test/status ledger maintained by prior agent sessions (self-reported; partially reproduced independently — see STRATEX_TEST_AND_BUILD_EVIDENCE.md).
test_reports/	JSON/screenshot artifacts from prior frontend UI test iterations; a .gitkeep 'd directory for future device-pilot evidence.

5. CENTCOM / Cortex / ATC / Reality Foundry / Core / Passport / Pro — what is and isn't evidenced here

- The string [CENTCOM](#) appears **only** as a document-header prefix ("CENTCOM DIRECTIVE H-00N: ...") on dozens of root-level spec markdown files (e.g. [PROJECT_OPPORTUNITY_SPEC.md:1](#)). It is a mission-issuing label in documentation, not a subsystem with code in this repository.
- [Cortex](#), [Core](#), and [Passport](#) appear as **named upstream systems this app talks about or defers to** — never as code owned by this repo:
- [backend/passport_projection.py](#) implements a **client-side adapter** with four provider modes ([development](#), [demo](#), [test](#), [production](#)). The [production](#) provider ([HttpProjectionProvider](#), line 287) calls an external URL from [HABITAT_PROJECTION_BASE_URL](#); if that env var is unset — which it is, everywhere in this repo, including CI — it **fails closed** with [ProjectionUnavailable](#) ([passport_projection.py:291-297](#)). No real Passport/Core/Cortex endpoint is configured, called successfully, or verified reachable anywhere in this codebase.
- [backend/server.py:475](#) contains a function literally named/commented [Mock STRATEX Core publishing approved scans/reports into the shared property record](#) ([server.py:460-476](#)) — i.e. the "Core" integration in the running app is explicitly a mock.
- **Conclusion on the directive's architecture claim** ("Core is embedded inside CENTCOM between Cortex and Passport; Pro and Habitat are standalone"): this repository provides **no evidence to confirm or deny** the internal CENTCOM/Cortex/Core/Passport topology, because none of those systems' code is present here. What this repo **does** confirm is that Habitat is architected as a standalone consumer of a Passport-shaped projection API and does not itself implement Cortex/Core/Passport. Whether "Stratex Pro" is genuinely a separate standalone codebase cannot be assessed — it is not present in this environment and is only mentioned in passing in product-spec markdown, never as a build target or integration in code.

6. Test credentials note

[backend/seed.py](#) seeds fixed demo accounts ([alex@stratexhabitat.com](#), [admin@stratexhabitat.com](#), [horizon@stratexhabitat.com](#), [peakbuild@stratexhabitat.com](#), [lonestar@stratexhabitat.com](#)) with the password [Demo123!/Admin123!](#) hardcoded in source and reused by the pytest integration suite ([backend/tests/conftest.py:31-33](#)). These are non-production, seed-only, low-entropy demo credentials — acceptable for a dev/demo seed, but they must never be reused for any real deployment, and their presence in source is a configuration-hygiene item (see [STRATEX_TECHNICAL_VALIDATION_REPORT.md §9](#)).

3. Field Data Blockers

STRATEX FIELD DATA BLOCKERS

What in this repository genuinely requires real DJI/LiDAR property-scan data to complete or validate, versus what is conventional software work that does not depend on field data. Evidence is cited to specific files.

1. What requires real property scans (cannot be responsibly completed with synthetic data alone)

Item	Current state	Why it needs real capture data	Evidence
M4E/M4T drone imagery ingestion	Not present in this repository at all.	No ingestion, parsing, or validation code for DJI M4E/M4T imagery exists here; this repo only consumes a hypothetical downstream "Passport" projection.	Absence confirmed by full-repo search; no dji , m4e , m4t references found anywhere in backend/ or frontend/ .
RTK metadata	Not present.	Same as above — no RTK/GPS-metadata parsing exists in this codebase.	Absence confirmed.
Reconstruction outputs (point clouds, meshes, orthomosaics)	Only a deterministic synthetic placeholder point cloud exists: _POINTCLOUD = b"STRATEX-H014B-PROOF-POINTCLOUD;x,y,z,r,g,b\n" * 900 (backend/reality/capture_proof.py:35-36). No mesh or orthomosaic generation/consumption code exists.	The upload/storage/checksum "machinery" around point clouds is real, but it has never ingested a real point cloud. Only real capture will prove the artifact/checksum/upload pipeline against real payload sizes, formats, and error modes.	backend/reality/capture_proof.py , backend/reality/capture_upload_service.py
Radiometric R-JPEG thermal data	Not present. No thermal-specific parsing, calibration, or radiometric conversion code exists anywhere in the repo.	Cannot be validated without real thermal captures; there is currently nothing to validate.	Absence confirmed by search.
Semantic geometry (rooms/openings/surfaces classification)	Backend has a spatial_service.py and SHARED_SPATIAL_DOMAIN_MODEL.md spec, but the only populated instance data is the fixed 12-entity, 2-frame deterministic fixture described in docs/h014a/H014A_REFERENCE_ROOM_CONTRACT.md and reproduced verbatim in test_result.md:463-465 (width 4.88 m, length 6.10 m, floor area 29.768 m ² , entity_count 12).	The data model/contract exists, but semantic accuracy, edge cases (irregular rooms, occlusion, multi-story), and classification reliability are entirely unproven against real scan variability.	backend/reality/spatial_service.py , backend/reality/fixtures.py

Item	Current state	Why it needs real capture data	Evidence
Property Object Graph / object extraction	docs/h014/PRODUCT_GRAPH_AND_MATERIAL_SELECTION.md is a specification. design.py 's "Product Graph" concepts operate on a hand-authored product library, not extracted objects from any scan.	No object-detection/extraction pipeline exists in this repo to extract real objects from any capture.	docs/h014/PRODUCT_GRAPH_AND_MATERIAL_SELECTION.md (spec only, per docs/h014/README.md:1 : "Specification only. Not implemented.")
Real-world measurement tolerances	docs/h014/PROPERTY_COORDINATE_AND_ALIGNMENT_STANDARD.md defines tolerance classes (spec), but H-014's own architecture record lists as an open "known unknown": "LiDAR device/SDK capability envelope and achievable PLANNING-tolerance residuals on real homes" (docs/h014/H014_MASTER_ARCHITECTURE_DECISION_RECORD.md:69).	Tolerance claims are policy/labels today, not measured accuracy. Only real scans compared against ground truth (tape measure, per docs/h014b3/H014B3_MANUAL_MEASUREMENT_SHEET.md) can validate them.	docs/h014/H014_MASTER_ARCHITECTURE_DECISION_RECORD.md , docs/h014b3/H014B3_MANUAL_MEASUREMENT_SHEET.md
Cortex findings/confidence	Not present in this repo; Cortex is an external system referenced only conceptually.	Cannot validate a system whose code is not in this repository at all.	N/A — out of repo scope
Core quantity/estimate validation	backend/pricebook.py is a governed price book with explicit <code>authoritative:false</code> provenance (per docs/h014/H014_REPOSITORY_AND_ARCHITECTURE_AUDIT.md:19); quantities come from user/homeowner inputs and design selections, not from measured/extracted geometry.	Estimate accuracy against real quantities/measurements from a real scan has never been tested.	backend/pricebook.py
Passport truth promotion	Explicitly and deliberately blocked in code: a completed capture yields only <code>DRAFT_CANDIDATE + MEASURED_EXISTING</code> ; promotion to <code>VERIFIED_EXISTING</code> requires "a separate authorized Core/Passport/professional review" not implemented here (docs/h014b/H014B_INDEX.md , "Truth boundary" section). backend/reality/audit_service.py emits <code>REALITY_TRUTH_PROMOTION_REJECTED</code> audit events specifically to prevent silent promotion.	By design, promotion is deferred to a real, externally-authorized review step that doesn't exist in this repo — inherently requires real captures and an external authority to exercise.	backend/reality/audit_service.py , docs/h014b/H014B_INDEX.md

Item	Current state	Why it needs real capture data	Evidence
Report accuracy (Comprehensive Property Intelligence Report)	backend/redaction.py and projects.py produce a "PIP" (Project Intent Package)/contractor package from homeowner-entered and demo/fixture data; no report in this repo has ever been generated from a real scan.	Report correctness against ground truth is unproven.	backend/projects.py , backend/redaction.py
Pro/Habitat real property synchronization	No "Stratex Pro" application exists in this environment/repo to synchronize with; Habitat's only sync-adjacent code is the demo sync_log collection (docs/h014/H014_REPOSITORY_AND_ARCHITECTURE_AUDIT.md:33-36) and the mocked "Core handshake" in backend/server.py:460-476 .	Cannot validate a cross-app sync that has no real counterpart app and only a mock handshake function in code.	backend/server.py:460-476
Before/after verification workflows	docs/habitat/H014C_ACCEPTANCE.md explicitly states Habitat prefers an "empty verified-finding state" for the demo and asserts "No fake meshes, rooms, dimensions, or LiDAR results." No before/after comparison against a real completed job has been run.	Requires two real captures of the same property (before/after a completed project) — cannot exist without field data by definition.	docs/habitat/H014C_ACCEPTANCE.md

2. What does NOT depend on field data and should still be finished

These items are ordinary software-engineering work, independent of drone/LiDAR capture, that this audit found incomplete or unverified:

- 1 **Frontend build/dependency pipeline is not verifiable at all in this audit** (network-blocked [yarn install](#)) — this needs to be fixed/verified in a network-enabled environment regardless of any field data. See [STRATEX_TEST_AND_BUILD_EVIDENCE.md §4](#).
- 2 **No lint/type-check enforcement**, despite [black/isort/flake8/mypy](#) being pinned dependencies ([backend/requirements.txt:22-25](#)) — wiring these into CI is pure engineering hygiene work.
- 3 **No security/SAST scanning workflow** exists ([.github/workflows/](#) has only functional test workflows) — independent of field data.
- 4 **Backend reality package coverage gaps** unrelated to hardware: [router.py](#), [schemas.py](#), [indexes.py](#), [capture_proof.py](#) show 0% in isolated process-local coverage ([STRATEX_TEST_AND_BUILD_EVIDENCE.md §3](#)) — closing this requires better test harnessing, not real scans.
- 5 **The two duplicate/unreconciled state machines** ([workflow.py](#)'s Steward workflow vs. [projects.py](#)'s [IDEA→...→SAVED_TO_PASSPORT](#)) are explicitly flagged as unreconciled in [docs/h014/H014_REPOSITORY_AND_ARCHITECTURE_AUDIT.md:67-70](#) ("Phase 16 defines ONE canonical... state machine... without rewriting either in this mission") — this is deferred design/refactoring work, not a field-data blocker.

- 6 **The entire H-014 Reality Studio architecture beyond H-014A/B is "specification only"** ([docs/h014/README.md:1](#)) — Interior/Exterior/Systems studio UIs, Room DNA/Property DNA projections, Living Digital Home layering, and the Product Graph are undesigned-in-code; these can and should be implemented against the existing fixture/demo data model before any real scan is required, so that the moment real captures arrive there is a working UI/data path to receive them.
- 7 [backend/server.py](#)'s **unconditional external storage-init call** at startup (network call to [integrations.emergentagent.com](#) even when `HABITAT_OBJECT_STORE_PROVIDER=fake`) is a configuration/robustness bug independent of field data — see [STRATEX_TEST_AND_BUILD_EVIDENCE.md](#) §2.
- 8 **Hardcoded demo credentials in [backend/seed.py](#)** should be parameterized/rotated as a config-hygiene practice before any real-property pilot, regardless of scan data.
- 9 **The native Swift capture module and host app are authored but never compiled/run in any CI observed by this audit** in a real macOS/Xcode environment from this session (workflow exists but requires a macOS runner not available here) — verifying that workflow actually runs green is engineering validation independent of having a physical LiDAR device in hand.

3. Bottom line

Everything that touches **real optical/RTK/thermal capture data, semantic object extraction from that data, measurement-tolerance validation, or Passport truth-promotion** is either unimplemented, spec-only, or backed exclusively by a small, fixed, deterministic synthetic fixture (12 entities / 2 frames / one demo room). Everything else — build pipeline health, lint/security gates, state-machine reconciliation, UI completion for already-modeled data, and fixing the mock external Core handshake — is ordinary software work that can and should proceed without waiting for field data.

4. Test and Build Evidence

STRATEX TEST AND BUILD EVIDENCE

All commands in this document were executed directly, in this session, inside

[/home/runner/work/stratex-habitat/stratex-habitat](#) on commit [6eeb7605a379f5954c7343909eaede57cfc0ea8b](#).

Where an independent re-run was not possible (no network, no macOS host, etc.), that limitation is stated explicitly — no result is inferred or assumed.

1. Backend dependency install

```
$ cd backend && pip install -r requirements.txt
...
ERROR: Could not find a version that satisfies the requirement emergentintegrations==0.2.0
ERROR: No matching distribution found for emergentintegrations==0.2.0
```

Result: FAILS as-authored. [emergentintegrations==0.2.0](#) is a private/non-PyPI package (confirmed by the repo's own CI workflow comment, [.github/workflows/backend-reality.yml:44-47](#): *"private/non-PyPI package used only by the optional live Gemini render path... Public GitHub Actions cannot install it"*). All 27 other pinned dependencies installed cleanly on Python 3.12. Excluding that one line (exactly as the project's own CI does) produces a clean, fully successful install.

2. Backend unit/integration test suite — independently reproduced

The suite in [backend/tests/](#) (16 files) is an **HTTP integration suite** ([backend/tests/conftest.py](#)) that requires a live MongoDB and a running [uvicorn](#) process — it is not a mocked/in-process unit suite. To get a real result, this session stood up the exact CI topology ([.github/workflows/backend-reality.yml](#)):

```
$ docker run -d --name habitat-ci-mongo -p 27017:27017 mongo:7.0
$ REACT_APP_BACKEND_URL=http://127.0.0.1:8001 MONGO_URL=mongodb://127.0.0.1:27017 \
  DB_NAME=habitat_ci JWT_SECRET=ci-h014b2-jwt-secret-not-for-production \
  HABITAT_ENV=test HABITAT_ENABLE_FIXTURES=true HABITAT_OBJECT_STORE_PROVIDER=fake \
  CORS_ORIGINS=* uvicorn server:app --host 127.0.0.1 --port 8001 --workers 1 &
backend ready after 4s

$ python -m pytest tests/ -rs -q
..... [ 21%]
..... [ 42%]
..... [ 63%]
..... [ 84%]
.....S..... [100%]
SKIPPED [1] tests/test_design_studio.py:204: Skip live Gemini render unless RUN_RENDER_TEST=1
338 passed, 1 skipped, 3 warnings in 8.19s
```

Result: PASS — 338 passed, 1 skipped, 0 failed (independently reproduced, not copied from [test_result.md](#)). The 1 skip is an intentionally gated live Gemini AI render test ([RUN_RENDER_TEST=1](#) not set) — a real external LLM call, correctly excluded from default CI. This is a materially larger count than the last self-reported figure in [test_result.md](#) (281 passed / 1 skipped at H-014A.2), consistent with additional H-014B/B.3/C tests added since. The self-reported historical counts in [test_result.md](#) are therefore plausible and directionally consistent with what this session verified live, but they were **not independently reproducible from those earlier commits** in this audit (only HEAD was tested) — treat historical pass counts in [test_result.md](#) as unverified secondary evidence, and the **338 passed** figure above as the only

primary evidence in this report.

A first-startup log line is notable and was captured directly:

```
ERROR - Storage init failed: ... Failed to resolve 'integrations.emergentagent.com'
```

This confirms `backend/server.py`'s object-storage client unconditionally attempts to reach a real external managed-storage endpoint (<https://integrations.emergentagent.com/objstore/api/v1/storage>) at startup regardless of the `HABITAT_OBJECT_STORE_PROVIDER=fake` env var used by CI/this test run — it fails soft (logs and continues) rather than crashing, but it is evidence of an unmanaged external dependency at boot (see technical report §9).

3. Backend coverage (reality package)

```
$ python -m pytest tests/ -q --cov=reality --cov-report=term-missing --cov-fail-under=0 -n 0
TOTAL      1862    945    49%
338 passed, 1 skipped, 2 warnings in 8.19s
```

Process-local coverage of the `reality/` package is **49%**, matching the CI workflow's own documented expectation that "plain pytest --cov against a separate uvicorn measures ~49% process-local" (`backend-reality.yml:12-14`). CI's authoritative $\geq 83\%$ figure requires combining this run with a coverage-instrumented `uvicorn` subprocess plus a second in-process unit run — that combination was **not reproduced in this session** (time-boxed); the 49% process-local number above is the only coverage figure this audit can attest to first-hand. Modules at 0% in the process-local run: `reality/router.py`, `reality/schemas.py`, `reality/indexes.py`, `reality/capture_proof.py` (these are exercised only via the HTTP layer inside the separate uvicorn process, which process-local `--cov` cannot attribute to).

4. Frontend dependency install / build

```
$ cd frontend && yarn install
...
error Error: getaddrinfo ENOTFOUND assets.emergent.sh
```

Result: BLOCKED — no network access to the required package/asset host in this sandbox. `yarn build` / `craco build` were **not attempted** because install did not complete; no frontend build result can be reported. This is an environment limitation, not a code defect, but it means the frontend build could not be independently validated in this audit.

Frontend automated tests: only 2 Jest test files exist

(`frontend/src/propertyVisualization/__tests__/PropertyVisualization.test.js`, `frontend/src/components/__tests__/Inspector.test.js`); neither could be run (same install blocker). Prior UI verification evidence in this repo is screenshot/JSON-based manual-agent testing under `test_reports/` (e.g. `test_reports/h014b1_browser/report.json`), not an automated CI-gated test run.

5. Lint / static analysis

```
$ find backend frontend -maxdepth 3 \( -name ".eslintrc*" -o -name "eslint.config.*" \
-o -name "ruff.toml" -o -name "pyproject.toml" -o -name ".flake8" \
-o -name "mypy.ini" -o -name "tsconfig.json" \)
(no results)
$ rg "\"eslintConfig\"|\"lint\"|\"ruff\"|\"flake8\"|\"mypy\"" backend frontend
(no results)
```

Result: no lint/type-check configuration or script exists in either `backend/` or `frontend/`, despite `black`, `isort`, `flake8`, and `mypy` all being listed in `backend/requirements.txt` (lines 22-25) — they are installed dependencies with **no invoked configuration, no CI step, and no committed config file**. This is a gap: linting/type-checking tooling is present

as a dependency but not wired into any enforceable gate.

6. Security / static-analysis checks

No SAST/dependency-scanning tool (e.g. `bandit`, `safety`, `pip-audit`, `npm audit` in CI, CodeQL workflow) is configured in `.github/workflows/` (only `backend-reality.yml` and `ios-capture.yml` exist, both functional/unit test workflows, not security scans). No security-scan evidence exists to report beyond what this audit's own tooling produces separately.

7. Existing CI-equivalent commands — reproduced results

Workflow	Trigger scope	This session's result
<code>backend-reality.yml</code>	push/PR touching <code>backend/**</code>	Reproduced. 338 passed / 1 skipped / 0 failed (§2 above); coverage combine step not reproduced (§3).
<code>ios-capture.yml</code>	push/PR touching <code>ios/**</code>	Not reproducible in this environment — requires <code>macos-latest</code> GitHub runner with Xcode/Swift/iOS Simulator SDK; no macOS host is available here. Per the workflow's own scope (<code>ios-capture.yml:1-11</code>), even a successful run only proves the Swift package compiles for the iOS Simulator SDK and passes portable (non-hardware) Guardian unit tests — it explicitly does not exercise physical LiDAR capture (<code>ios-capture.yml:9-11</code> : <i>"This is NOT physical LiDAR execution (still UNEXECUTED)"</i>).

8. Summary pass/fail table

Check	Status	Evidence
Backend dependency install (as pinned)	FAIL (1 unreachable private package)	§1
Backend dependency install (CI's exclusion method)	PASS	§1
Backend integration test suite	PASS — 338 passed, 1 skipped, 0 failed	§2
Backend <code>reality</code> package coverage (process-local)	49% (informational; CI's authoritative combined gate is ≥83%, not reproduced)	§3
Frontend dependency install	BLOCKED (no network to <code>assets.emergent.sh</code>)	§4
Frontend build	NOT RUN (blocked by above)	§4
Frontend automated tests	NOT RUN (blocked by above)	§4
Backend lint/type-check	NOT CONFIGURED (tools installed, unused)	§5
Security/SAST scanning	NOT CONFIGURED	§6
<code>ios-capture.yml</code> (Swift build/test)	NOT REPRODUCIBLE HERE (needs macOS); by design never proves physical capture	§7